

TSI

Tendereo Sustainability Institute

Architecture Technique

Documentation complete de l'architecture du projet

Genere le 19/06/2026

Table des matieres

1. 1. Presentation generale
2. 2. Stack technique
3. 3. Structure des fichiers
4. 4. Core Application
5. 5. Modeles de donnees
6. 6. Blueprints et routes
7. 7. Templates HTML
8. 8. Assets statiques
9. 9. Securite
10. 10. Tests
11. 11. Base de donnees

1. Presentation generale

TSI (Tendereo Sustainability Institute) est une plateforme web complete pour la finance durable en Afrique. Elle permet aux utilisateurs d'investir dans des Actifs de Revenus Fonciers (ARF), d'acheter et vendre des droits contractuels sur une marketplace, d'utiliser un simulateur d'investissement, de payer via mobile money (Orange Money et MTN MoMo), de participer a des reunions en ligne, et de publier des articles.

L'application est developpee en Python avec le framework Flask, utilise MySQL comme base de donnees principale (SQLite en fallback), et Bootstrap 5 pour le design responsive.

2. Stack technique

Composant	Technologie	Version
Langage	Python	3.12
Framework	Flask	3.0.0
ORM	Flask-SQLAlchemy	3.1.1
Auth	Flask-Login	0.6.3
Forms	Flask-WTF	1.2.1
Migrations	Flask-Migrate	4.0.5
Email	Flask-Mail	0.9.1
i18n	Flask-Babel	4.0+
BDD principale	MySQL (PyMySQL)	1.1.0
BDD fallback	SQLite	Integre
Frontend	Bootstrap	5.3.2
Icons	Bootstrap Icons	1.11.2
Video	Jitsi Meet (iFrame)	Gratuit
Paielements	Orange Money + MTN MoMo	API REST

3. Structure des fichiers

```
think-tank/
├── run.py                # Point d'entree
├── .env                  # Variables d'environnement
├── requirements.txt      # Dependances Python
├── babel.cfg             # Configuration i18n
├── instance/
│   └── tsi_platform.db   # SQLite (fallback)
├── translations/         # Traductions FR/EN
├── app/
│   ├── __init__.py      # Factory Flask + blueprint registration
│   ├── config.py         # Classes de configuration
│   ├── extensions.py     # Instances des extensions
│   ├── models/          # Modeles de donnees
│   │   ├── __init__.py
│   │   ├── user.py
│   │   ├── arf.py
│   │   ├── marketplace.py
│   │   ├── transaction.py
│   │   ├── meeting.py
│   │   └── article.py
│   └── auth/             # Blueprint authentication
```

```

├── ├── __init__.py
│   ├── routes.py
│   └── forms.py
├── admin/                                # Blueprint administration
│   ├── __init__.py
│   ├── routes.py
│   └── forms.py
├── main/                                # Blueprint pages publiques
│   ├── __init__.py
│   └── routes.py
├── arf/                                  # Blueprint investissement ARF
│   ├── __init__.py
│   ├── routes.py
│   └── forms.py
├── marketplace/                         # Blueprint marketplace
│   ├── __init__.py
│   ├── routes.py
│   └── forms.py
├── simulator/                           # Blueprint simulateur
│   ├── __init__.py
│   └── routes.py
├── member/                              # Blueprint espace membre
│   ├── __init__.py
│   └── routes.py
├── payment/                             # Blueprint paiements
│   ├── __init__.py
│   ├── routes.py
│   ├── orange_money.py
│   └── mtn_momo.py
├── blog/                                # Blueprint blog
│   ├── __init__.py
│   └── routes.py
├── reseau/                              # Blueprint reseau (reunions + articles)
│   ├── __init__.py
│   ├── routes.py
│   └── forms.py
├── static/
│   ├── css/style.css
│   └── js/main.js
├── templates/
│   ├── base.html
│   ├── auth/ (5 fichiers)
│   ├── admin/ (4 fichiers)
│   ├── main/ (4 fichiers)
│   ├── arf/ (5 fichiers)
│   ├── marketplace/ (5 fichiers)
│   ├── simulator/ (1 fichier)
│   ├── member/ (3 fichiers)
│   ├── payment/ (2 fichiers)
│   ├── reseau/ (10 fichiers)
│   └── errors/ (2 fichiers)
├── tests/
│   └── test_platform.py

```

4. Core Application

4.1 run.py - Point d'entree

Fichier minimal qui cree l'application via la factory et lance le serveur de developpement.
Contient juste create_app() et app.run(debug=True).

4.2 app/__init__.py - Factory Flask

Coeur de l'application. Contient :

- create_app(config_name) : factory qui configure l'app, initialise les extensions, enregistre les 10 blueprints
- get_locale() : selectionne la langue (FR par default, changeable via session)
- set_security_headers() : ajoute X-Content-Type-Options, X-Frame-Options, X-XSS-Protection
- set_lang(language) : route /set-language/<language> pour changer la langue
- load_user(user_id) : charge l'utilisateur pour Flask-Login
- Gestion des erreurs 404 et 500 avec templates customises
- Creation automatique des tables avec db.create_all() au demarrage

4.3 app/config.py - Configuration

Trois classes de configuration chargees depuis .env :

- Config : SECRET_KEY, DATABASE_URL, MAIL_*, ADMIN_SECRET_PATH
- DevelopmentConfig : DEBUG=True
- ProductionConfig : DEBUG=False

4.4 app/extensions.py - Extensions

Instances partagees initialisees une fois :

- db = SQLAlchemy() : ORM pour MySQL/SQLite
- login_manager = LoginManager() : gestion de l'authentification
- mail = Mail() : envoi d'emails
- migrate = Migrate() : migrations de base de donnees

5. Modeles de donnees

8 tables MySQL avec relations definies via SQLAlchemy ORM.

5.1 Table users

Colonne	Type	Description
id	Integer PK	Identifiant unique
email	String(255) UNIQUE	Email de l'utilisateur
password_hash	String(256)	Mot de passe chiffre (Werkzeug)
first_name	String(100)	Prenom
last_name	String(100)	Nom
phone	String(20)	Telephone (optionnel)
role	String(10)	admin ou user
is_active	Boolean	Compte actif ou bloque
language	String(5)	fr ou en
created_at	DateTime	Date de creation
last_login	DateTime	Derniere connexion

5.2 Table arf_assets

Colonne	Type	Description
id	Integer PK	Identifiant unique
name	String(255)	Nom de l'ARF
description	Text	Description complete
sector	String(100)	Secteur d'activite
country	String(100)	Pays
total_value	Numeric(15,2)	Valeur totale du fonds
unit_price	Numeric(10,2)	Prix par tikec
total_units	Integer	Nombre total de tikecs
units_sold	Integer	Tikecs vendus
expected_return	Numeric(5,2)	Rendement attendu (%)
risk_level	String(10)	low/medium/high
status	String(10)	active/closed/pending
created_by	FK -> users.id	Admin createur
created_at	DateTime	Date de creation

5.3 Table subscriptions

Colonne	Type	Description
id	Integer PK	Identifiant unique
user_id	FK -> users.id	Utilisateur
arf_id	FK -> arf_assets.id	ARF souscrit
units	Integer	Nombre de tikecs
amount	Numeric(15,2)	Montant total
payment_method	String(20)	orange_money ou mtn_momo
phone_number	String(20)	Numero de telephone
transaction_ref	String(100)	Reference de transaction
status	String(10)	pending/completed/failed
created_at	DateTime	Date de souscription

5.4 Table marketplace_offers

Colonne	Type	Description
id	Integer PK	Identifiant unique
user_id	FK -> users.id	Utilisateur createur
title	String(255)	Titre de l'offre
description	Text	Description
contract_type	String(100)	Type de contrat
value	Numeric(15,2)	Valeur du droit
deadline	Date	Echeance (optionnel)
status	String(10)	available/sold/expired
created_at	DateTime	Date de creation

5.5 Table transactions

Colonne	Type	Description
id	Integer PK	Identifiant unique
user_id	FK -> users.id	Utilisateur
subscription_id	FK -> subscriptions.id	Souscription liee
type	String(20)	subscription/deposit/withdrawal
amount	Numeric(15,2)	Montant
currency	String(5)	XOF
payment_method	String(20)	orange_money ou mtn_momo
phone_number	String(20)	Telephone
reference	String(100) UNIQUE	Ref unique (TSI-YYYYMMDD-XXXXXXX)
status	String(15)	pending/processing/completed/failed/cancelled
created_at	DateTime	Date de creation
updated_at	DateTime	Date de mise a jour

5.6 Table meetings

Colonne	Type	Description
id	Integer PK	Identifiant unique
title	String(255)	Titre de la reunion
description	Text	Description
room_id	String(36) UNIQUE	UUID pour le lien Jitsi
host_id	FK -> users.id	Organisateur
status	String(10)	active/ended/cancelled
max_participants	Integer	Limite (defaut 50)
created_at	DateTime	Date de creation
ended_at	DateTime	Date de fin

5.7 Table meeting_participants

Colonne	Type	Description
id	Integer PK	Identifiant unique
meeting_id	FK -> meetings.id	Reunion
user_id	FK -> users.id	Participant
role	String(10)	host/moderator/participant
joined_at	DateTime	Heure d'entree

left_at	DateTime	Heure de sortie
is_kicked	Boolean	Expulse ou non

5.8 Table articles

Colonne	Type	Description
id	Integer PK	Identifiant unique
user_id	FK -> users.id	Auteur
title	String(255)	Titre
content	Text	Contenu
category	String(50)	Categorie
status	String(10)	pending/published/archived/ rejected
admin_comment	Text	Commentaire admin
views	Integer	Nombre de vues
created_at	DateTime	Date de creation
published_at	DateTime	Date de publication

6. Blueprints et routes

L'application compte 10 blueprints et 57 routes au total.

Blueprint	Prefixe	Routes	Description
auth	/auth	5	Authentification, inscription, verification admin
admin	/admin	6	Dashboard admin, gestion utilisateurs
main	/	4	Pages publiques (accueil, a propos, contact, guide)
arf	/arf	8	Catalogue ARF, souscription, CRUD admin
marketplace	/marketplace	7	Offres, publication, gestion admin
simulator	/simulator	1	Simulateur d'investissement
member	/member	3	Dashboard membre, portefeuille, transactions
payment	/payment	7	Paielements Orange/MTN, webhooks
blog	/blog	2	Blog (coming soon)
reseau	/reseau	13	Reunions live, articles, moderation

6.1 Niveaux d'accès

Niveau	Nombre	Routes
Public	15 routes	Accueil, a propos, catalogue ARF, marketplace, simulateur, blog, contact, guide, login, inscription
Utilisateur connecte	18 routes	Dashboard membre, portefeuille, transactions, souscription, paiement, reunions, articles
Organisateur (host)	4 routes	Terminer reunion, expulser, participants
Admin	20 routes	Gestion utilisateurs, CRUD ARF, moderation articles, webhooks

7. Templates HTML

42 templates Jinja2 organisés par blueprint, s'étendant tous depuis base.html.

7.1 base.html - Template de base

Contient la structure commune :

- Navbar responsive Bootstrap 5 avec liens dynamiques
- Bandeau clignotant 'Comment utiliser TSI' (effets de lumière)
- Système de flash messages
- Footer avec liens et copyright
- Toggle dark/light mode (localStorage)
- Switcher de langue FR/EN
- Blocs extensibles : extra_css, content, scripts

7.2 Templates par blueprint

Dossier	Nombre	Fichiers
auth/	5	login, register, admin_verify, admin_code, admin_register
admin/	4	dashboard, users/list, users/detail, users/edit
main/	4	home, about, contact, how_to_use
arf/	5	index, detail, admin_create, admin_list, admin_edit
marketplace/	5	index, detail, create, my_offers, admin_list
simulator/	1	index
member/	3	dashboard, portfolio, transactions
payment/	2	status, history
reseau/	10	index, meetings (list, create, room, participants), articles (list, detail, create, my), admin/articles
errors/	2	404, 500

8. Assets statiques

8.1 style.css

Fichier CSS complet (377 lignes) avec :

- Variables CSS pour les themes clair/sombre
- Couleurs TSI : Vert #1B4332, Or #D4A843
- Style du hero, cartes, cartes statistiques, tables
- Boutons (primary, gold, outline-gold)
- Navbar, footer, sidebar, dropdowns
- Animations (fadeInUp, pulse, glow)
- Mode sombre avec variables CSS
- Scrollbar personnalisée
- Responsive (media queries mobile/tablette)

8.2 main.js

JavaScript minimal (47 lignes) :

- Toggle dark/light mode avec persistance localStorage
- Auto-dismiss des alerts Bootstrap après 5 secondes

9. Securite

Mesure	Implementation
Hachage mots de passe	Werkzeug generate_password_hash / check_password_hash
Protection brute-force (login)	5 tentatives max, verrouillage 15 min
Protection brute-force (code admin)	3 tentatives max, verrouillage 15 min + deconnexion
Double verification admin	Login + code 12345 avant acces au panneau
Decorateur admin_required	login_required + role check + session verification
Headers de securite	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection
CSRF Protection	Flask-WTF CSRF token sur tous les formulaires
Secret admin URL	Lien secret pour creer des comptes admin (pas visible publiquement)
Auto-protection admin	Un admin ne peut pas se supprimer ou se bloquer lui-meme

10. Tests

54 tests dans tests/test_platform.py couvrant :

Categorie	Nombre	Sujet
Routes publiques	11 tests	Accueil, a propos, contact, login, inscription, ARF, marketplace, simulateur, blog, admin secret
Authentification	7 tests	Inscription, login, acces dashboard/portfolio/transactions, logout
Administration	12 tests	Inscription admin, login, verification code, dashboard, gestion utilisateurs, ARF/marketplace admin
ARF	7 tests	Formulaire creation, creation, catalogue, detail, admin list, edit, update
Marketplace	5 tests	Formulaire creation, creation, catalogue, detail, mes offres
Simulateur	1 test	Calcul du simulateur
Paieement	1 test	Historique paiements
Modeles/BDD	10 tests	Verification des donnees en base

11. Base de donnees

Configuration :

- Principale : MySQL via PyMySQL (mysql+pymysql://tsi_user:***@localhost/tsi_platform)
- Fallback : SQLite (instance/tsi_platform.db) quand MySQL n'est pas disponible
- Creation automatique : db.create_all() au demarrage de l'application
- Migrations : Flask-Migrate (Alembic) pour les mises a jour schema